

TopoJSON

TopoJSON extends GeoJSON by encoding *topology*, that is, relationships between individual shapes. Instead of storing the coordinates of each shape, TopoJSON stores line segments (called *arcs*) that are used to build the shapes, removing duplicates that occur when two shapes share a common border. This eliminates redundancy and results in a file that is much smaller and more efficient. TopoJSON makes it easier to compute neighboring shapes, merge all shapes into one and create a mesh from shape outlines. This can considerably reduce the number of objects in a Web page, making it more memory efficient.

TopoJSON files aren't as popular as GeoJSON, but you can easily generate one using command-line or online tools. D3 supports TopoJSON through a JavaScript library with methods that generate GeoJSON data from TopoJSON.

The following code reveals the structure of a simple TopoJSON file (`data/africa/africa-simple.topojson`) generated from a GeoJSON file (`data/africa/africa-simple.geojson`). It has half the size of the original file:

```
{
  "type": "Topology",
  "objects": {
    "africa-simple": {
      "type": "GeometryCollection",
      "geometries": [
        { "type": "MultiPolygon", "arcs": [[[8],[9],[10],[11]],[[12]]] },
        { "type": "MultiLineString", "arcs": [[0,1],[2]],
          "properties": { "name": "Nile" } },
        { "type": "LineString", "arcs": [3], "properties": { "name": "Congo" } },
        // ... more LineStrings
        { "type": "Point", "coordinates": [3,7], "properties": { "name": "Lagos" } },
        // ... more Points
      ]
    }
  },
  "arcs": [
    [ [31,31], ..., [33,16] ],
    [ [33,16], ..., [33,0] ],
    // more arc arrays...
  ],
  "bbox": [-17,-35,51,37]
}
```

TopoJSON is available from github.com/topojson, where it is distributed as a collection of three different modules that can be installed via *npm* or accessed using a *CDN*:

- *topojson-server*: contains command line tools to generate TopoJSON from GeoJSON. Install this package if you have GeoJSON files you wish to convert to TopoJSON.
- *topojson-simplify*: functions for additional simplification.
- *topojson-client*: functions to convert TopoJSON into GeoJSON, merge geometries, create meshes, compute neighbors and other tasks.

You can also install the full *topojson* bundle, which includes all three modules.

To use TopoJSON with *d3-geo* we only need *topojson-client*. It can be included in an HTML file using a CDN such as *skypack*:

```
import * as topojson from 'https://cdn.skypack.dev/topojson-client@3'
```

Its main functions are listed in *Table 1*.

Function	Description
<code>feature(data, object)</code>	Returns the GeoJSON <i>feature</i> for a specified <i>object</i> from the topology <i>data</i> , which are properties of the <i>objects</i> member in a TopoJSON file. Their name is application-dependent. The result is a standard GeoJSON <i>FeatureCollection</i> where each object in the geometries property is included in a separate <i>Feature</i> .
<code>merge(data, geometries)</code>	Merges all objects in the geometries array into a single object. The result is a single GeoJSON <i>MultiPolygon</i> representing the union of all <i>Polygon</i> and <i>MultiPolygon</i> objects, with common borders removed.
<code>mesh(data, object, filter)</code>	Creates a <i>mesh</i> from the outlines and borders of the object. If <i>object</i> is not specified, it will create a mesh from the entire topology. The result is a single GeoJSON <i>MultiLineString</i> . A filter can be provided to remove specific arcs from the mesh (see example below).
<code>neighbors(geometries)</code>	Returns an array containing a list of arrays, each representing an object. The elements of each inner array contain the <i>indexes</i> of its neighbors in the outer <i>neighbors</i> array (see example below).

Table 1 – Main functions from the *client-topojson* module.

This section contains some code snippets that demonstrate how to use the functions above, but also try running the complete examples. Their full code is available in the TopoJSON/ directory.

Obtaining GeoJSON features

A TopoJSON file contains at least one object of type *Topology* with an *objects* member, which may contain one or more *geometry* objects. This property is not an array: *geometry*

objects are *referenced by name* and you need to know the name of the geometry object you wish to use. Most files contain a single object and you can discover its name by simply inspecting the source code. You can also run the following code and obtain a list:

```
d3.json('../data/africa-simple.topojson').then(function(data) {
  const geoms = [];
  for(const key in data.objects) {
    geoms.push(key);
  }
  const keys = geoms.join(); // a list of geometry object names
})
```

The data file above contains a single geometry object: "africa-simple".

To use a TopoJSON file in code that expects GeoJSON features, you need to extract the *features* object stored in the geometry object using the `topojson.feature()` function. It takes two arguments: the TopoJSON data and a selected geometry object and returns a GeoJSON *FeatureCollection* containing a GeoJSON *Feature* for each geometry object, including the properties object. After loading this file, you can extract the GeoJSON data as shown in the following code (See [TopoJSON/1-features-africa-simple.html](#)):

```
d3.json('../data/africa-simple.topojson').then(function(data) {
  const object = data.objects["africa-simple"];
  const geodata = topojson.feature(data, object); // FeatureCollection
})
```

Now you can render the data just as before, obtaining the array of features with `geodata.features` and using it as your dataset:

```
svg.selectAll("path.continent")
  .data(geodata.features)
  .join("path").attr("class", "continent")
  .style("fill", (d,i) => isLine(d) ? 'none' : color(i))
  .attr("d", geoPath);
```

This should produce the same result as [GeoJSON/3-lines-africa.html](#).

To explore TopoJSON's `merge()`, `mesh()` and `neighbors()` operations in the next examples we shall use a considerably larger data file: `data/africa/africa.topojson`. It doesn't contain cities or rivers but includes *Polygon* and *MultiPolygon* shapes of several adjacent countries and properties, which are better to illustrate how TopoJSON works. See the full code in [TopoJSON/2-features-africa-countries.html](#). It contains practically the same code as the last example but renders several countries on the screen in different colors.

A second version of these examples using simple squares instead of country borders uses the `data/geojson/border-example.topojson` file, which is very simple. An example of the `features()` function with this file is available in [TopoJSON/3-features-shapes.html](#).

Merging geometries

The `merge()` function takes an array of geometries as the second argument, returning a single GeoJSON *MultiPolygon* with the *union* of all the objects in a geometry collection:

```
const polygon = topojson.merge(data, object.geometries)
```

Since a single object is returned, you can bind it to a path using `selection.datum()`:

```
svg.append("path").datum(polygon).attr("d", geoPath);
```

The result is a map with a single shape created from all shapes combined, and no internal divisions. In this example you will see the entire continent of Africa, without any country borders. The full code is available in [TopoJSON/4-merge-africa.html](#). See also [5-merge-shapes.html](#) that contains the same code using another TopoJSON file containing simple shapes.

You don't have to merge the entire dataset. Since this data file contains a `subregion` property that groups several countries, we can use it to filter which geometries to include:

```
const geometries = data.objects['africa'].geometries;  
    .filter(d => d.properties.subregion=="North Africa");  
const polygon = topojson.merge(data, geometries)
```

This was used in the interactive map shown in *Figure 1*. It draws a semitransparent merged shape over a group of objects when the mouse hovers any member of the group. It only contains the shapes with the same `subregion` property as the clicked shape.

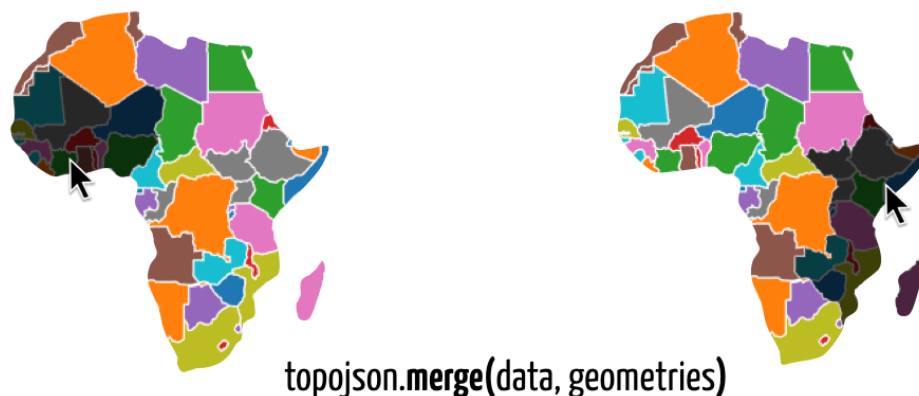


Figure 1 – Using `topojson.merge()` to draw a *MultiPolygon* containing the union of several shapes.
Code: [TopoJSON/6-merge-africa-regions.html](#)

Creating a mesh

The `topojson.mesh()` function returns a mesh of the entire topology, which includes outlines and borders between shapes.

```
const geomesh = topojson.mesh(data);
```

A mesh is a *LineString* rendered as a single path. It should not have any fill style, only stroke.

You can also filter the objects you want to mesh based on whether they share or not the same *arcs*. This can be used to separate the outline of a topology (e.g. the coastlines of the continent) from the internal borders. The filter takes two arguments that will be identical if an arc is used by a single geometry, and different if they are shared.

```
const borders = topojson.mesh(data, data.objects['africa'], (a,b) => a !== b)
const outline = topojson.mesh(data, data.objects['africa'], (a,b) => a === b )
```

The effect is illustrated in *Figure 2*. You can see the complete code is in the files *TopoJSON/7-mesh-africa.html*, *9-mesh-africa-borders.html* and *10-mesh-africa-outline.html*.

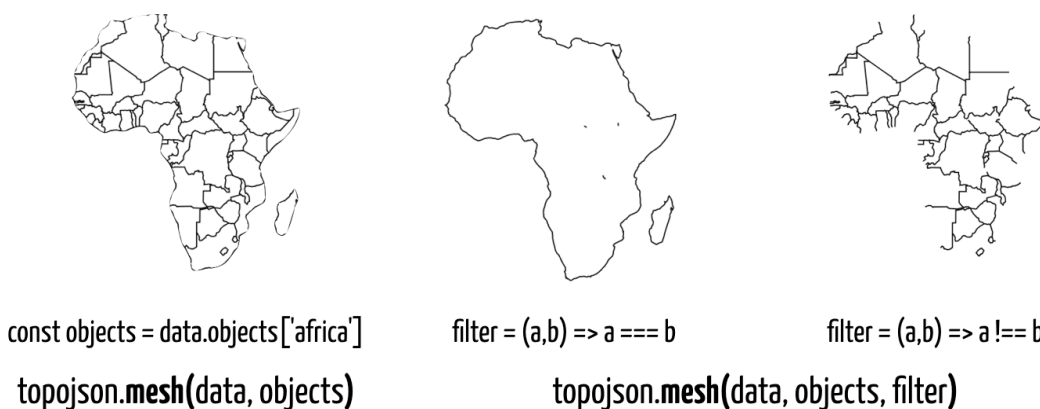


Figure 2 – Using `topojson.mesh()` to create outlines and a mesh of internal borders.
Code: *TopoJSON/7-mesh-africa.html*, *9-mesh-africa-borders.html* and *10-mesh-africa-outline.html*.

See also the examples using a data file with simple shapes in *TopoJSON/8-mesh-shapes.html* and *11-mesh-shapes-outline.html*.

In a large dataset, the use of `merge()` and `mesh()` combined with drawing the shapes on a Canvas can significantly improve the performance of an interactive animated map.

Discovering neighbors

Since adjacent shapes share common borderlines in *TopoJSON*, it's easy to compute their neighbors. The `topojson.neighbors()` function receives an array of geometries and returns an array of arrays with the neighbors of each shape:

```
const neighbors = topojson.neighbors(object.geometries);
```

Each array inside the main `neighbors` array represents a shape, and their contents are array-indexes of the shapes that have common borders with them. This is made clearer in *Figure 3*, which compares an array of neighbors with the rendered shapes. You can see the code and run this example in *TopoJSON/13-neighbors-shapes.html*:

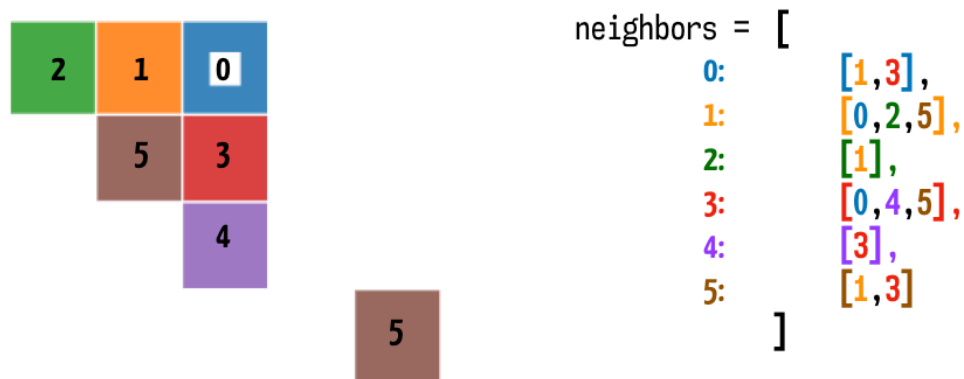


Figure 3 – A neighbors array generated with *topojson.neighbors()*.
Code: *TopoJSON/13-neighbors-shapes.html*.

With the neighbors array, you can select a specific shape by its index and then retrieve the indexes of its neighbors. In the following code (from *TopoJSON/12-neighbors-africa.html*) event handlers were added to highlight a shape and its neighbors on mouse hover:

```
svg.selectAll("path.country")
  .on('mouseover', showNeighbors)
  .on('mouseout', showAll);

function showNeighbors(d,i) {
  d3.selectAll('.country')
    .classed('faded', (k,j) => !(map.neighbors[i].includes(j) || k === d))
    .classed('highlighted', k => k === d);
}

function showAll(d) {
  d3.selectAll('.country')
    .classed('faded', false)
    .classed('highlighted', false);
}
```

These styles change the colors and opacity of the shapes when activated:

```
.faded {
  opacity: .3;
}

.highlighted {
  fill: black;
  stroke-width: 2
}
```

The result is shown in *Figure 4*, where hovering over a country will reveal the countries that have borders with it.

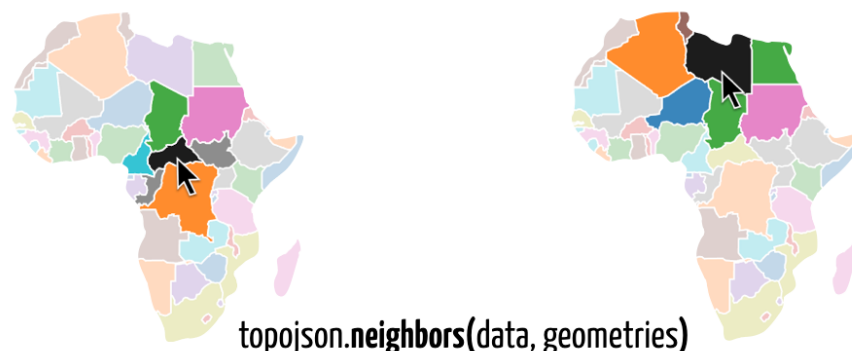


Figure 4 – Using *topojson.neighbors()* to discover shapes that have common borders.
Code: *TopoJSON/12-neighbors-africa.html*.

Converting GeoJSON to TopoJSON

Many GIS applications and online tools convert GeoJSON to TopoJSON. *MapShaper* (mapshaper.org) is very simple to use and supports big GeoJSON files. You can also install the *topojson-server* in your computer with NPM:

```
npm install -g topojson-server
```

Now you can call the *geo2topo* command-line application and transform GeoJSON files into TopoJSON. The simplest way to do it is to use:

```
geo2topo source.geojson -o target.topojson
```

The transformation can be tuned by applying a quantization parameter. This may be necessary if your input data is messy and there are gaps between adjacent borders. The parameter is a number, usually a power of ten. Very low values will produce a pixelated map. Higher values allow more points per dimension but also use more memory. Typical values are 1000, 1e4, 1e6, etc.

```
geo2topo source.geojson -q 1e4 -o target.topojson
```

There are many other functions available, mostly for simplification. They can be used to filter parts of a topology, calculate areas and bounding boxes, remove coordinates and apply transforms. Command-line versions of *mesh()* and *merge()* are also available and can be used to pre-process a file before using it.